

PLANO MESTRE DO PROJETO

Sistema Corporativo de Gestão de Estoque e Almoxarifado Inteligente — WMS

Versão do planejamento: 1.0

Data-base tecnológica: Agosto de 2026

Tipo: Aplicação Web + Mobile + API + Banco de Dados

Arquitetura: Modular Monolith + Clean Architecture

Objetivo: Projeto acadêmico e de portfólio com arquitetura e regras próximas às utilizadas em sistemas corporativos reais.

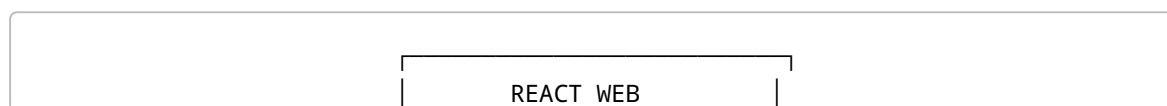
1. Visão geral

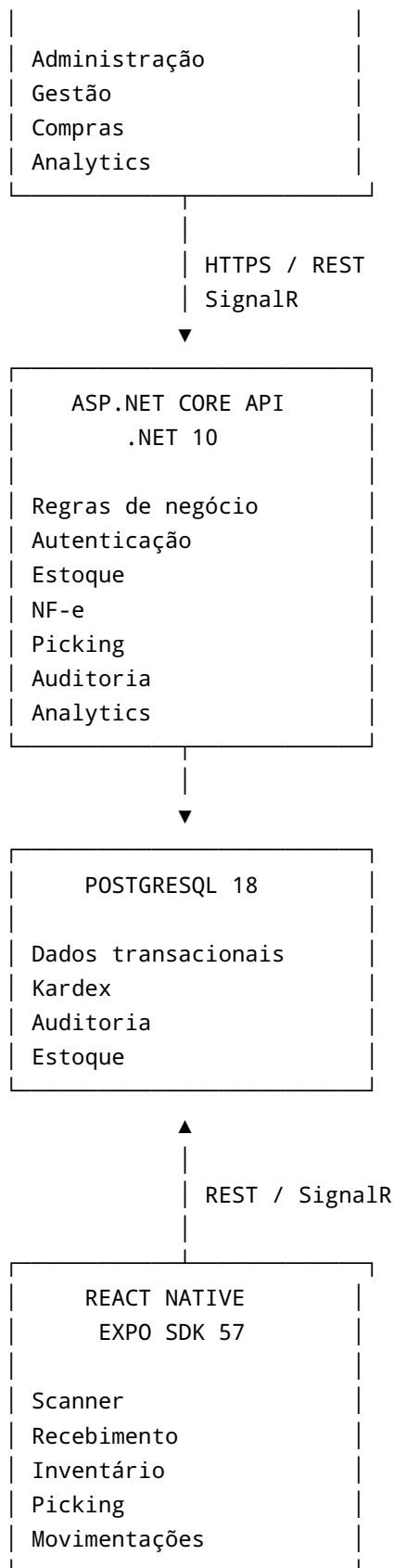
O projeto consiste no desenvolvimento de um **Warehouse Management System — WMS**, ou Sistema de Gestão de Armazéns e Almoxarifados.

O sistema será responsável por controlar todo o ciclo operacional de materiais dentro de uma empresa, desde o recebimento de mercadorias até a expedição, incluindo:

- produtos;
- fornecedores;
- lotes;
- validade;
- almoxarifados;
- galpões;
- endereçamento físico;
- recebimento;
- importação de NF-e;
- movimentações;
- transferências;
- inventários;
- reservas;
- pedidos;
- picking;
- FEFO;
- custos;
- auditoria;
- dashboards;
- alertas;
- rastreabilidade.

O projeto será dividido em três aplicações principais:





2. Problemas que o sistema pretende resolver

Empresas que ainda controlam estoques por planilhas ou processos manuais enfrentam problemas como:

- divergências entre estoque físico e sistema;
- perda de materiais;
- produtos esquecidos;
- produtos vencidos;
- ausência de rastreabilidade;
- erros de digitação;
- retrabalho;
- dificuldade em localizar mercadorias;
- baixa velocidade de separação;
- movimentações não identificadas;
- ausência de histórico;
- custos incorretos;
- compras desnecessárias;
- falta de informações para decisões;
- inventários demorados;
- dificuldade em determinar responsabilidades.

O sistema deverá reduzir esses problemas através de automação, validação e rastreabilidade.

3. Objetivo principal

Construir uma plataforma empresarial capaz de:

Controlar, rastrear, movimentar, armazenar, inventariar e analisar materiais dentro de um ou mais almoxarifados utilizando interfaces Web e Mobile integradas em tempo real.

4. Objetivos secundários

O projeto também deve demonstrar conhecimentos em:

- C#;
- .NET;
- APIs REST;
- arquitetura de software;
- Domain-Driven Design em nível pragmático;
- Clean Architecture;
- PostgreSQL;
- modelagem relacional;
- transações;
- concorrência;

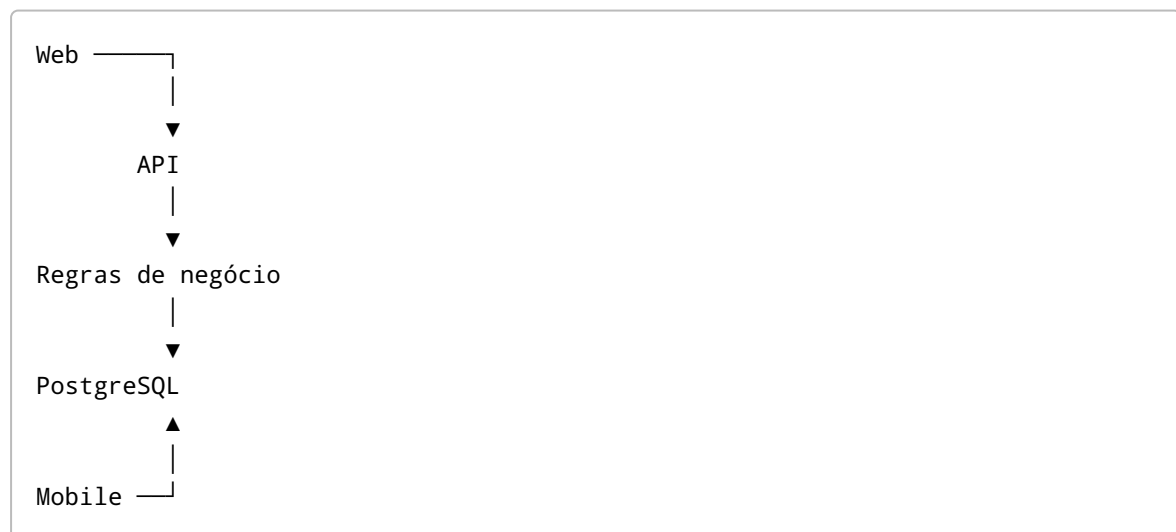
- React;
- TypeScript;
- React Native;
- desenvolvimento mobile;
- Docker;
- autenticação;
- autorização;
- testes;
- WebSockets;
- processamento de XML;
- Git;
- GitHub;
- CI/CD;
- observabilidade;
- segurança;
- algoritmos;
- experiência do usuário.

5. Princípios arquiteturais

O projeto seguirá os seguintes princípios.

5.1 Uma única fonte de verdade

Todas as alterações de estoque deverão passar pelo backend.



Nem Web nem Mobile alterarão estoque diretamente.

5.2 Modular Monolith

O sistema **não começará utilizando microserviços**.

Será desenvolvido como um Modular Monolith.

```
WMS
|
├─ Identity
├─ Catalog
├─ Suppliers
├─ Warehouse
├─ Inventory
├─ Receiving
├─ Orders
├─ Picking
├─ InventoryCount
├─ Costing
├─ NFe
├─ Audit
├─ Notifications
└─ Analytics
```

Cada módulo possuirá responsabilidades claramente definidas.

6. O que NÃO será utilizado inicialmente

Para impedir overengineering, não entraremos inicialmente com:

- microserviços;
- Kubernetes;
- RabbitMQ;
- Kafka;
- Redis;
- Event Sourcing completo;
- múltiplos bancos;
- Elasticsearch;
- arquitetura offline-first;
- IA;
- machine learning;
- mapa 3D;
- roteirização avançada.

Esses recursos poderão existir futuramente se houver uma necessidade real.

7. Stack tecnológica oficial

Backend

C

Linguagem principal do backend.

.NET 10 LTS

Framework principal do servidor.

ASP.NET Core Web API

Responsável pelos endpoints REST.

Entity Framework Core 10

ORM principal.

Responsável por:

- mapeamento;
- migrations;
- queries;
- transações;
- persistência.

Npgsql

Provider PostgreSQL para EF Core.

FluentValidation 12+

Validação de Commands e DTOs.

Exemplos:

```
Quantidade > 0  
Código obrigatório  
EAN válido  
Data válida  
Usuário existente  
Localização ativa
```

ASP.NET Core Identity

Base para gerenciamento de:

- usuários;
- senhas;
- roles;
- tokens.

JWT

Autenticação das aplicações.

Refresh Tokens

Renovação segura das sessões.

SignalR

Comunicação em tempo real.

Utilizado para:

- atualização de estoque;
- alertas;
- dashboards;
- inventários;
- notificações.

OpenAPI

Documentação formal dos endpoints.

Scalar

Interface visual para exploração/teste da API.

Serilog

Logs estruturados.

OpenTelemetry

Observabilidade e rastreamento.

8. Decisão sobre MediatR

MediatR **não** será dependência obrigatória.

Utilizaremos handlers próprios através de interfaces.

Exemplo:

```
CreateProductCommand
    ↓
CreateProductHandler
    ↓
Domain
    ↓
Database
```

Isso mantém CQRS conceitualmente sem criar uma dependência desnecessária.

9. Banco de dados

PostgreSQL 18

Banco relacional principal.

Será responsável por garantir não somente persistência, mas também integridade.

Utilizaremos:

- primary keys;
 - foreign keys;
 - unique constraints;
 - check constraints;
 - índices;
 - transações;
 - isolamento;
 - locking;
 - consultas agregadas;
 - JSONB quando realmente necessário.
-

10. Frontend Web

React 19

Aplicação administrativa.

TypeScript

Tipagem estática.

Vite

Build tool.

Tailwind CSS

Estilização.

shadcn/ui

Base de componentes visuais.

TanStack Query

Server state.

Responsável por:

- cache;
- refetch;
- mutations;
- invalidation;
- loading;
- retry.

React Hook Form

Gerenciamento de formulários.

Zod

Schemas e validação da interface.

Zustand

Somente para estados globais de UI quando necessários.

TanStack Query não deverá ser substituído por Zustand para dados vindos do servidor.

Recharts

Gráficos e dashboards.

SignalR Client

Atualizações em tempo real.

React Three Fiber / Three.js

Somente em fase futura para mapa 3D.

11. Aplicativo Mobile

React Native

Framework mobile.

Expo SDK

Ecossistema e ferramentas.

TypeScript

Linguagem.

Expo Router

Navegação.

TanStack Query

Comunicação e cache da API.

expo-camera

Scanner através da câmera.

Suporte planejado para:

- EAN-13;
- EAN-8;
- Code-128;
- QR Code.

expo-haptics

Feedback através de vibração.

APIs de áudio

Feedback sonoro.

expo-secure-store

Armazenamento seguro de credenciais/tokens.

12. Infraestrutura

Docker

Containerização.

Docker Compose

Orquestração local.

Containers previstos:

```
wms-api  
wms-web  
wms-postgres
```

Ambiente de desenvolvimento poderá incluir:

```
pgAdmin
```

O pgAdmin não deverá obrigatoriamente estar presente em produção.

13. Estrutura do repositório

Será utilizado inicialmente um **monorepo**.

```
wms/  
|  
├─ backend/  
│  └─ src/  
│    └─ tests/  
├─ web/  
├─ mobile/  
└─ infra/  
|
```

```
| docs/
|
| scripts/
|
| .github/
|
| docker-compose.yml
| .editorconfig
| .gitignore
| CONTRIBUTING.md
| SECURITY.md
| LICENSE
| README.md
```

14. Backend

Estrutura inicial:

```
backend/
|
| src/
|   |
|   | Wms.Api/
|   | Wms.Application/
|   | Wms.Domain/
|   | Wms.Infrastructure/
|
| tests/
|   | Wms.UnitTests/
|   | Wms.IntegrationTests/
|   | Wms.ArchitectureTests/
```

15. Wms.Domain

Não deverá depender das outras camadas.

Contém:

- Entities;
- Value Objects;
- Enums;
- Domain Events;
- Domain Exceptions;
- regras puras;

- serviços de domínio.

Exemplo:

```
Product
Warehouse
Location
InventoryLot
StockMovement
```

16. Wms.Application

Contém os casos de uso.

```
Products/
  Create/
  Update/
  Search/

Inventory/
  Receive/
  Transfer/
  Adjust/

Picking/
  Create/
  Confirm/
```

Cada operação poderá utilizar:

```
Command
Validator
Handler
Result
```

17. Wms.Infrastructure

Contém implementações técnicas.

Exemplos:

- Entity Framework;
- PostgreSQL;

- autenticação;
 - persistência;
 - SignalR;
 - importação NF-e;
 - armazenamento de arquivos;
 - serviços externos.
-

18. Wms.Api

Responsável pela camada HTTP.

Contém:

- endpoints;
- authentication;
- authorization;
- middleware;
- filters;
- dependency injection;
- OpenAPI;
- rate limiting;
- ProblemDetails.

A API não deverá conter regras complexas de estoque.

19. Principais módulos

Identity

Responsável por:

- usuários;
 - autenticação;
 - roles;
 - permissões;
 - tokens;
 - dispositivos;
 - sessões.
-

Catalog

Responsável por:

- produtos;
- categorias;

- marcas;
 - unidades;
 - códigos de barras;
 - SKUs.
-

Suppliers

Responsável por:

- fornecedores;
 - produtos por fornecedor;
 - unidades de compra;
 - códigos externos.
-

Warehouse

Responsável pelo ambiente físico.

Warehouse

↓

Zone

↓

Aisle

↓

Rack

↓

Level

↓

Bin

Inventory

Responsável por:

- estoque;
 - lotes;
 - saldos;
 - movimentações;
 - reservas;
 - transferências.
-

Receiving

Responsável pelo recebimento.

NFe

Responsável pela leitura e interpretação de XML.

Orders

Responsável pelas solicitações de saída.

Picking

Responsável pela separação.

InventoryCount

Responsável por inventários.

Costing

Responsável por custos.

Audit

Responsável por rastreabilidade administrativa.

Notifications

Responsável por alertas.

Analytics

Responsável por indicadores.

20. Usuários

Perfis iniciais:

Perfil	Responsabilidade
Administrador	Configuração do sistema
Gestor	Gestão do estoque
Comprador	Compras e NF-e
Estoquista	Movimentação física
Separador	Picking
Auditor	Inventário e auditoria

21. RBAC

O sistema deverá utilizar permissões.

Exemplos:

```
products.read
products.create
products.update
products.deactivate

inventory.read
inventory.receive
inventory.transfer
inventory.adjust

inventory-count.create
inventory-count.execute
inventory-count.approve

orders.read
orders.create

picking.execute

nfe.import

reports.read

users.manage
```

22. Autenticação

Fluxo:

```
Login
↓
Validação
↓
Access Token
+
Refresh Token
```

Access Token deverá possuir vida curta.

Refresh Token deverá possuir:

- expiração;
- rotação;
- revogação;
- identificação da sessão.

23. Segurança dos tokens

Web

Preferencialmente:

```
Access Token
→ memória da aplicação

Refresh Token
→ HttpOnly Secure Cookie
```

Evitar armazenar credenciais sensíveis permanentemente em `localStorage`.

Mobile

Utilizar armazenamento seguro do sistema operacional.

24. Cadastro de produtos

Produto deverá suportar:

- ID;
- SKU;
- nome;
- descrição;
- categoria;
- marca;
- unidade;
- códigos de barras;
- peso;
- dimensões;
- ativo/inativo;
- controla lote;
- controla validade;
- estoque mínimo;
- estoque máximo;
- ponto de reposição.

25. SKU e Barcode

SKU representa o identificador interno.

Exemplo:

SSD-KIN-NV3-1TB

Um produto poderá possuir múltiplos códigos de barras.

Product
↓
ProductBarcode

26. Unidades de medida

Exemplos:

UN
CX
KG

G
LT
ML
MT

Será possível estabelecer conversões.

Exemplo:

1 CX = 24 UN

27. Fornecedores

Dados previstos:

- razão social;
- nome fantasia;
- CNPJ;
- inscrição estadual;
- endereço;
- contato;
- telefone;
- e-mail;
- status.

28. Produtos do fornecedor

Será necessária uma tabela de mapeamento.

SupplierProduct

SupplierId
ProductId
SupplierSku
SupplierDescription
PurchaseUnit
ConversionFactor

Assim:

Fornecedor :
ABC-991

Sistema:
SSD-KINGSTON-1TB

29. Estrutura física

Cada posição deverá possuir código único.

Exemplo:

A-03-07-02-04

Representando:

Zona A
Corredor 03
Rack 07
Nível 02
Posição 04

30. Capacidade das posições

Uma posição poderá possuir:

- limite de peso;
- largura;
- altura;
- profundidade;
- volume;
- tipos permitidos.

Isso permitirá futuramente:

sugestão inteligente de armazenagem.

31. Lotes

Estrutura:

InventoryLot

```
Id
ProductId
LotNumber
ManufactureDate
ExpirationDate
SupplierId
```

Nem todo produto precisará controlar lote.

32. Validade

Produtos poderão possuir:

```
TracksExpiration = true
```

Quando ativado, movimentações deverão possuir lote válido.

33. Coração do sistema: Inventory Ledger

O sistema não deverá controlar estoque apenas através de:

```
Product.Quantity
```

Isso seria inadequado.

Cada operação deverá gerar uma movimentação.

34. InventoryMovement

Estrutura conceitual:

```
InventoryMovement

Id
MovementNumber

ProductId
WarehouseId
LocationId
LotId
```

MovementType

Quantity

PreviousBalance
NewBalance

UnitCost
TotalCost

ReferenceType
ReferenceId

ReasonCode
Notes

UserId
DeviceId

CreatedAt

35. Tipos de movimentação

Exemplos:

```
RECEIPT  
ISSUE  
TRANSFER_IN  
TRANSFER_OUT  
ADJUSTMENT_IN  
ADJUSTMENT_OUT  
PICKING  
RETURN  
REVERSAL  
INVENTORY_ADJUSTMENT
```

36. Kardex imutável

Movimentações consolidadas não serão excluídas nem alteradas.

Exemplo:

```
+100 RECEBIMENTO  
-20 PICKING
```

-5 AJUSTE

75 SALDO

Caso exista erro:

+5 ESTORNO

em vez de modificar a movimentação anterior.

37. StockBalance

Também existirá uma projeção de saldo otimizada.

StockBalance

ProductId

WarehouseId

LocationId

LotId

OnHandQuantity

ReservedQuantity

AvailableQuantity

Regra:

Available =

OnHand - Reserved

38. Por que Ledger + Balance?

Ledger responde:

O que aconteceu?

Balance responde:

Quanto temos agora?

Assim obtemos:

- rastreabilidade;
- performance.

39. Estoque negativo

Não será permitido por padrão.

Regras existirão tanto na aplicação quanto no banco.

Exemplo conceitual:

```
CHECK (on_hand_quantity >= 0)
CHECK (reserved_quantity >= 0)
CHECK (reserved_quantity <= on_hand_quantity)
```

40. Concorrência

Problema:

```
Saldo = 1

Operador A consulta → 1
Operador B consulta → 1

A retira 1
B retira 1
```

O sistema deverá impedir:

```
Saldo = -1
```

Utilizaremos:

- transações;
- locks apropriados;
- operações atômicas;
- constraints.

Dependendo da operação, poderá ser utilizado:

```
SELECT ... FOR UPDATE
```

ou atualização condicional atômica.

41. Idempotência

Endpoints críticos deverão aceitar:

```
Idempotency-Key
```

Exemplo:

O celular envia uma saída.

A rede cai antes de receber a resposta.

O celular tenta novamente.

Sem idempotência:

```
-10  
-10
```

Com idempotência:

```
Solicitação já processada.
```

42. Recebimento

Fluxo:

```
Fornecedor  
↓  
Documento  
↓  
Recebimento  
↓  
Conferência  
↓  
Lote
```

↓
Custo
↓
Entrada
↓
Put-away
↓
Endereçamento

43. Importação NF-e

Fluxo:

Upload XML
↓
Validação
↓
Parser
↓
Dados NF-e
↓
Fornecedor
↓
Itens
↓
Product Matching
↓
Revisão
↓
Recebimento

44. O XML não deverá movimentar estoque automaticamente

Após importar:

NF-e 12345

Fornecedor: ACME Distribuidora

Produto NF-e

Produto WMS

ABC0001	Parafuso 10mm	✓
ABC0002	Arruela 10mm	✓
XYZ321	Não encontrado	!

O usuário deverá confirmar o mapeamento.

Somente depois ocorrerá o recebimento.

45. Prevenção de NF-e duplicada

Armazenaremos:

- chave de acesso;
- número;
- série;
- CNPJ emitente;
- hash do XML.

Uma NF-e já processada não poderá gerar nova entrada.

46. Campos fiscais úteis

O módulo deverá estar preparado para armazenar:

- chave de acesso;
- CNPJ;
- número;
- série;
- data;
- NCM;
- CFOP;
- CEST;
- GTIN;
- unidade comercial;
- quantidade;
- valor unitário;
- descontos;
- frete;
- tributos informativos.

47. Compliance fiscal

O WMS **não deverá afirmar ser um sistema contábil ou fiscal plenamente certificado.**

O projeto implementará práticas de:

- rastreabilidade;
- custo;
- armazenamento de informações fiscais;
- Kardex.

Validações contábeis definitivas exigiriam especialistas do domínio.

48. Custo médio

Exemplo:

```
100 × R$ 10 = R$ 1.000
+
100 × R$ 14 = R$ 1.400
-----
200 unidades
R$ 2.400

Custo médio = R$ 12
```

A implementação deverá levar em conta as regras definidas pelo projeto.

49. Costing

Será criado módulo próprio.

```
Costing/
├─ CalculateAverageCost
├─ InventoryValuation
└─ CostHistory
```

Não colocar cálculos contábeis diretamente em Controllers.

50. Reservas

Exemplo:

Físico:
100

Reservado:
30

Disponível:
70

Quando um pedido é separado:

Available → diminui
Reserved → aumenta

Na expedição:

Reserved → diminui
OnHand → diminui

51. Pedidos

Pedido possuirá:

Order

Id
Number
Status
RequestedBy
CreatedAt

e:

OrderItem

Product
Quantity

52. Estados do pedido

Possível state machine:

```
DRAFT
↓
RELEASED
↓
ALLOCATED
↓
PICKING
↓
PICKED
↓
CHECKED
↓
SHIPPED
```

Também:

```
CANCELLED
```

53. Picking

O backend determina:

- produto;
- quantidade;
- endereço;
- lote.

O aplicativo orienta o operador.

54. FEFO

Para produtos com validade:

```
First Expired
First Out
```

Exemplo:

L001 → 10/09/2026
L002 → 10/12/2026
L003 → 05/05/2027

Prioridade:

L001
L002
L003

55. FIFO

Produtos sem validade poderão utilizar:

First In
First Out

quando necessário.

56. Scanner mobile

Fluxo:

Tarefa:
Pegue 5 unidades

Local:
A-01-03-02

Operador lê a localização.

✓ LOCAL CORRETO

Depois lê o produto.

✓ PRODUTO CORRETO

Produto incorreto:

× PRODUTO INCORRETO

57. Feedback operacional

O aplicativo utilizará:

- cores;
- vibração;
- áudio;
- mensagens grandes.

O operador não deverá precisar interpretar telas complexas enquanto trabalha.

58. Inventário

Existirão inventários:

- gerais;
- parciais;
- por endereço;
- por categoria;
- por produto;
- rotativos.

59. Contagem cega

Durante a primeira contagem:

Produto:
SSD Kingston

Quantidade:
[]

O sistema NÃO mostrará a quantidade esperada.

Depois:

Sistema: 80
Contado: 77
Diferença: -3

60. Recontagem

Divergências poderão exigir:

Primeira contagem
↓
Divergência
↓
Recontagem
↓
Aprovação
↓
Ajuste

61. Transferências

Fluxo:

Origem
↓
Transferência
↓
Destino

A transferência deverá possuir rastreabilidade completa.

62. Auditoria

Registrar:

UserId
Action
Entity
EntityId
Before

After
Timestamp
IpAddress
Device
CorrelationId

63. AuditLog

Exemplo:

21/08/2026 10:42

Usuário:
Operador 03

Ação:
INVENTORY_ADJUSTMENT

Produto:
SSD Kingston 1TB

Antes:
100

Depois:
98

Motivo:
Divergência de inventário

64. Motivos de ajuste

Ajustes exigirão motivo.

Exemplos:

DAMAGE
LOSS
EXPIRATION
COUNT_DIFFERENCE
RETURN

SYSTEM_CORRECTION
OTHER

65. Dashboard

Indicadores:

- valor total do estoque;
- quantidade de SKUs;
- estoque disponível;
- itens abaixo do mínimo;
- produtos em ruptura;
- itens próximos ao vencimento;
- ocupação do galpão;
- recebimentos do dia;
- movimentações;
- pedidos pendentes;
- picking em andamento;
- divergências de inventário.

66. Curva ABC

O sistema poderá classificar produtos em:

A
B
C

Utilizando valor de consumo dentro de determinado período.

Não limitar a implementação simplesmente ao valor armazenado.

67. Giro de estoque

Indicador:

quantidade/valor consumido em relação ao estoque médio.

68. Aging

Identificação de estoque parado.

Exemplo:

```
0-30 dias
31-60 dias
61-90 dias
90-180 dias
180+ dias
```

69. Alertas

Sistema poderá gerar:

- estoque mínimo;
- estoque zerado;
- vencimento próximo;
- lote vencido;
- divergência de inventário;
- pedido atrasado;
- localização lotada.

70. SignalR

Eventos previstos:

```
StockUpdated
InventoryMovementCreated
LowStockDetected
ExpirationAlertCreated
ReceivingCompleted
InventoryCountUpdated
PickingUpdated
OrderStatusChanged
```

71. Interface Web

Menu principal:

Dashboard

Estoque

- └─ Visão geral
- └─ Kardex
- └─ Lotes
- └─ Movimentações
- └─ Transferências

Cadastros

- └─ Produtos
- └─ Categorias
- └─ Fornecedores
- └─ Unidades

Armazém

- └─ Galpões
- └─ Endereços
- └─ Ocupação
- └─ Mapa

Recebimentos

- └─ Recebimentos
- └─ Importar NF-e

Pedidos

- └─ Pedidos
- └─ Picking

Inventários

Relatórios

Alertas

Administração

- └─ Usuários
- └─ Perfis
- └─ Permissões
- └─ Auditoria
- └─ Configurações

72. Dashboard visual

Deverá possuir:

- KPIs;

- gráficos;
 - tabelas;
 - filtros de período;
 - alertas;
 - indicadores de tendência.
-

73. Páginas Web

Telas previstas:

1. Login
 2. Dashboard
 3. Produtos
 4. Novo Produto
 5. Produto Detalhado
 6. Categorias
 7. Fornecedores
 8. Fornecedor Detalhado
 9. Almoxarifados
 10. Localizações
 11. Estoque Atual
 12. Kardex
 13. Movimentações
 14. Transferência
 15. Lotes
 16. Validades
 17. Importação NF-e
 18. Recebimento
 19. Pedidos
 20. Pedido Detalhado
 21. Picking
 22. Inventários
 23. Contagem
 24. Relatórios
 25. Curva ABC
 26. Auditoria
 27. Alertas
 28. Usuários
 29. Roles
 30. Permissões
 31. Configurações.
-

74. Responsividade

Web deverá funcionar principalmente em:

- desktop;

- notebook;
- tablet.

O mobile operacional será atendido pelo aplicativo React Native.

75. UX

Regras:

- formulários objetivos;
 - carregamento visível;
 - erros claros;
 - confirmação em operações críticas;
 - atalhos;
 - tabelas com filtros;
 - pesquisas rápidas;
 - feedback imediato;
 - acessibilidade.
-

76. Aplicativo Mobile

Menu:

Início

Recebimento

Armazenagem

Movimentar

Picking

Inventário

Consultar Produto

Tarefas

77. Home mobile

Mostrar:

- operador;
 - tarefas;
 - recebimentos;
 - pickings;
 - inventários;
 - alertas.
-

78. Consulta por scanner

Operador lê:

7891234567890

Aplicativo mostra:

Produto
SKU
Descrição
Quantidade
Lotes
Endereços
Validade

79. Device registration

Dispositivos poderão ser cadastrados.

Device

Id
Name
Platform
LastSeen
Enabled

Movimentações poderão registrar o dispositivo.

80. Mobile inicialmente ONLINE

O aplicativo exigirá conexão com o servidor.

Não teremos sincronização offline no MVP.

Motivo:

estoque exige consistência.

81. API REST

Convenção:

```
/api/v1/
```

82. Endpoints — Products

Exemplos:

```
GET    /api/v1/products
GET    /api/v1/products/{id}
POST   /api/v1/products
PUT    /api/v1/products/{id}
DELETE /api/v1/products/{id}
```

Preferencialmente produtos serão desativados em vez de fisicamente excluídos quando possuírem histórico.

83. Inventory

```
GET    /api/v1/inventory
GET    /api/v1/inventory/products/{id}

POST   /api/v1/inventory/receipts
POST   /api/v1/inventory/issues
POST   /api/v1/inventory/transfers
POST   /api/v1/inventory/adjustments
```

84. Movements

```
GET /api/v1/inventory/movements
GET /api/v1/inventory/movements/{id}
```

Nenhum:

```
DELETE /inventory/movements/{id}
```

para movimentação consolidada.

85. NF-e

```
POST /api/v1/nfe/import
GET /api/v1/nfe/{id}
POST /api/v1/nfe/{id}/confirm
```

86. Inventários

```
POST /api/v1/inventory-counts

POST /api/v1/inventory-counts/{id}/start
POST /api/v1/inventory-counts/{id}/count
POST /api/v1/inventory-counts/{id}/recount
POST /api/v1/inventory-counts/{id}/close
```

87. API responses

Erros seguirão um formato consistente utilizando Problem Details.

Exemplo:

```
{
  "type": "inventory/insufficient-stock",
  "title": "Estoque insuficiente",
  "status": 409,
```

```
"detail": "Existem apenas 5 unidades disponíveis."  
}
```

88. Paginação

Listas grandes deverão utilizar:

```
page  
pageSize  
sort  
filter
```

Não devolver milhares de produtos de uma só vez.

89. Pesquisa

Suportar buscas por:

- SKU;
 - nome;
 - barcode;
 - lote;
 - localização.
-

90. Banco — principais tabelas

```
users  
roles  
permissions  
user_roles  
role_permissions  
  
products  
product_barcodes  
categories  
brands  
units  
  
suppliers  
supplier_products  
  
warehouses
```

```
zones
aisles
racks
levels
bins

inventory_lots
stock_balances
inventory_movements
stock_reservations

receivings
receiving_items

nfe_documents
nfe_items

orders
order_items

pickings
picking_items

inventory_counts
inventory_count_items

audit_logs

notifications
```

91. IDs

Entidades poderão utilizar:

```
UUID
```

com geração adequada.

Números amigáveis serão separados.

Exemplo:

```
ID interno:
018f...
```

Número:
REC-2026-000014

92. Datas

Todas as datas técnicas deverão ser armazenadas de maneira consistente.

Preferencialmente UTC para timestamps técnicos.

A apresentação utilizará timezone da empresa.

93. Soft delete

Não utilizar indiscriminadamente.

Entidades de cadastro poderão ser:

Active = false

Dados transacionais importantes não serão apagados.

94. Integridade

O banco deverá possuir constraints.

Nunca confiar somente no frontend.

95. Índices

Criar índices para:

- SKU;
- barcode;
- chave NF-e;
- lot number;
- product_id;
- warehouse_id;
- location_id;
- timestamps;
- status.

Índices deverão ser definidos com base nas consultas reais.

96. Uploads

Arquivos aceitos inicialmente:

.xml

Uploads deverão validar:

- tipo;
- tamanho;
- conteúdo;
- estrutura.

Nunca confiar somente na extensão.

97. Armazenamento de documentos

XML original poderá ser preservado para auditoria.

No início pode utilizar armazenamento persistente local controlado.

Futuramente poderá utilizar um object storage S3-compatible.

98. Segurança

Implementar:

- HTTPS;
 - autenticação;
 - RBAC;
 - validação;
 - CORS restritivo;
 - rate limiting;
 - limites de upload;
 - hashes seguros de senha;
 - proteção de secrets;
 - logs seguros;
 - tokens expiráveis;
 - refresh token rotation.
-

99. Secrets

Nunca versionar:

```
senha do banco
JWT secret
connection strings de produção
certificados
API keys
```

Utilizar:

```
environment variables
dotnet user-secrets
```

100. Logs

Logs deverão possuir:

- timestamp;
- nível;
- aplicação;
- endpoint;
- correlation ID;
- usuário quando apropriado;
- contexto.

Nunca registrar:

- senha;
- access token;
- refresh token;
- dados secretos.

101. Correlation ID

Cada requisição possuirá identificador.

Isso permitirá:

```
Web
↓
```



```
API
↓
Database
↓
SignalR
```

ser rastreado como uma operação.

102. Health Checks

Endpoints:

```
/health/live
/health/ready
```

Verificando:

- API;
 - PostgreSQL;
 - dependências críticas.
-

103. Observabilidade

Estrutura preparada para:

- logs;
- metrics;
- traces.

Tecnologia:

```
OpenTelemetry
```

104. Testes Backend

Unit Tests

Testar regras isoladas.

Exemplos:

cálculo FEFO
custo médio
validação
reservas
transições de status

105. Integration Tests

Testar:

API
+
PostgreSQL real em container

Preferencialmente utilizando Testcontainers.

106. Testes de concorrência

Obrigatórios para estoque.

Exemplo:

Saldo inicial: 5

Request A: retirar 4
Request B: retirar 4

Resultado:

uma pode ser aceita
a outra deve falhar

107. Testes Web

Utilizar:

- Vitest;
- Testing Library;
- Playwright.

108. Testes Mobile

Testes de lógica/componentes e, nas fases posteriores, fluxos E2E.

Maestro poderá ser utilizado para automação de jornadas importantes.

109. Testes críticos

Casos obrigatórios:

- estoque não negativo;
 - NF-e duplicada;
 - produto inexistente;
 - barcode duplicado;
 - usuário sem permissão;
 - reserva superior ao disponível;
 - picking com item errado;
 - endereço errado;
 - lote vencido;
 - duplicação causada por retry;
 - movimentação simultânea;
 - inventário divergente.
-

110. Git

Branch principal:

main

Branch de integração, caso o grupo prefira:

develop

Features:

feat/inventory-receipt
feat/products
feat/mobile-scanner

Correções:

fix/stock-concurrency

111. Conventional Commits

Exemplos:

```
feat: add product registration
fix: prevent negative stock
test: add inventory concurrency tests
docs: update architecture
refactor: extract costing service
chore: configure docker compose
```

112. Pull Requests

Todo PR deverá informar:

- o que mudou;
 - por quê;
 - como testar;
 - screenshots quando houver UI;
 - migrations existentes;
 - riscos.
-

113. Code Review

Idealmente nenhum integrante aprova sozinho mudanças críticas próprias.

Áreas prioritárias:

- estoque;
 - autenticação;
 - migrations;
 - NF-e;
 - custos.
-

114. GitHub Actions

Pipeline:

Push / Pull Request

↓

Backend Build

↓

Backend Tests

↓

Web Lint

↓

Web Build

↓

Web Tests

↓

Mobile Typecheck

↓

Docker Build

115. Qualidade de código

Configurar:

- formatter;
- lint;
- analyzers;
- TypeScript strict;
- nullable reference types;
- warnings;
- `.editorconfig`.

116. Documentação

Pasta:

docs/

Deverá conter:

architecture/
database/
api/
business-rules/
adr/
diagrams/

```
nfe/  
deployment/
```

117. ADR

Architecture Decision Records.

Exemplo:

```
ADR-001-use-modular-monolith.md  
ADR-002-use-postgresql.md  
ADR-003-use-ledger-plus-balance.md  
ADR-004-mobile-online-first.md  
ADR-005-no-mediator.md
```

Isso dará grande valor profissional ao repositório.

118. Diagramas

Produzir:

- C4 Context;
- C4 Container;
- C4 Component;
- ER Diagram;
- fluxo de recebimento;
- fluxo de estoque;
- fluxo de picking;
- autenticação;
- implantação.

119. Docker Compose

Ambiente inicial:

```
Docker Compose  
  
├─ PostgreSQL  
├─ API  
└─ Web
```

Mobile roda no dispositivo.

120. Ambientes

Ter:

```
Development
Testing
Production
```

Nunca utilizar banco de desenvolvimento como produção.

121. Seed

Desenvolvimento poderá possuir dados de demonstração.

Exemplos:

- administrador;
- produtos fictícios;
- fornecedores;
- galpão;
- endereços.

Nunca utilizar seed com senhas reais.

122. Backup

Docker volume NÃO é backup.

Estratégia deverá incluir:

```
PostgreSQL
↓
pg_dump
↓
Backup
↓
armazenamento separado
```

123. Restore

Procedimento de recuperação deverá ser documentado e testado.

124. Requisitos não funcionais

O sistema deverá possuir:

Segurança

Proteção contra acesso não autorizado.

Confiabilidade

Movimentações não podem desaparecer.

Consistência

Saldo precisa refletir as operações.

Performance

Consultas comuns devem responder rapidamente.

Escalabilidade

Arquitetura deverá permitir evolução futura.

Auditabilidade

Operações relevantes rastreáveis.

Usabilidade

Fluxos do estoque devem ser simples.

Manutenibilidade

Código modular e testável.

Observabilidade

Problemas devem poder ser diagnosticados.

125. Metas iniciais de performance

Não são SLA comercial, mas referências de engenharia.

API comum:

p95 < 500 ms

quando executada em infraestrutura adequada e sem operações externas lentas.

Operações de scanner deverão possuir percepção de resposta quase imediata na rede local.

126. Mapa 2D

Antes do 3D deverá existir visualização lógica.

Exemplo:



A01	██████████	80%
A02	████	30%
A03	██████████	100%
B01	██	20%
B02	██████	60%

127. Mapa 3D — fase futura

Tecnologias:

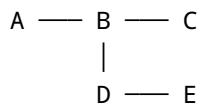
Three.js
React Three Fiber

Representará:

- corredor;
 - racks;
 - níveis;
 - posições;
 - ocupação.
-

128. Rota inteligente — fase futura

O galpão poderá ser modelado como grafo.



Algoritmos possíveis:

- Dijkstra;
- A*;
- nearest neighbour;
- heurísticas de picking.

129. Heatmap

Futuramente mostrar:

- posições mais acessadas;
- produtos de maior giro;
- áreas congestionadas.

130. Inteligência futura

Somente depois do sistema estar estável poderão ser avaliados:

- previsão de demanda;
- previsão de ruptura;
- sugestão de compra;
- slotting inteligente;
- detecção de anomalias;
- otimização de picking.

131. Escopo do MVP

O MVP possuirá:

- autenticação;
- usuários;
- roles;
- permissões básicas;

- produtos;
- categorias;
- unidades;
- fornecedores;
- galpões;
- endereçamento;
- lotes;
- estoque;
- Kardex;
- saldo;
- entradas;
- saídas;
- transferências;
- ajustes;
- inventário básico;
- auditoria;
- Docker;
- testes críticos.

Se isso estiver 100% funcionando, já existe um produto demonstrável.

132. MVP 2

Adicionar:

- NF-e;
 - recebimento estruturado;
 - conversões;
 - custo médio;
 - put-away;
 - mobile;
 - scanner.
-

133. MVP 3

Adicionar:

- pedidos;
 - reservas;
 - picking;
 - FEFO;
 - conferência;
 - expedição;
 - SignalR.
-

134. Versão avançada

Adicionar:

- analytics;
 - curva ABC;
 - aging;
 - giro;
 - alertas;
 - mapas;
 - otimização.
-

135. Fases oficiais de desenvolvimento

FASE 0 — Engenharia

Antes do código:

- nome;
- identidade;
- requisitos;
- casos de uso;
- arquitetura;
- ERD;
- ADRs;
- organização GitHub.

Definition of Done

Arquitetura aprovada pelo grupo.

FASE 1 — Fundação

Criar:

- repositório;
 - solução .NET;
 - React;
 - PostgreSQL;
 - Docker;
 - estrutura;
 - CI.
-

FASE 2 — Identity

Criar:

- login;
 - usuários;
 - roles;
 - permissions;
 - JWT;
 - refresh;
 - audit básico.
-

FASE 3 — Catálogo

Criar:

- produtos;
 - categorias;
 - unidades;
 - barcode;
 - fornecedores.
-

FASE 4 — Warehouse

Criar:

- warehouse;
 - zone;
 - aisle;
 - rack;
 - level;
 - bin.
-

FASE 5 — Inventory Core

Criar:

- lot;
- StockBalance;
- InventoryMovement;
- receipt;
- issue;
- transfer;
- adjustment;
- reservations;
- concorrência;

- idempotência.

Essa é a fase mais importante do projeto.

FASE 6 — Inventory Count

Criar:

- inventário;
 - contagem cega;
 - divergência;
 - recontagem;
 - aprovação;
 - ajuste.
-

FASE 7 — NF-e + Receiving

Criar:

- upload;
 - parser;
 - validação;
 - fornecedor;
 - product matching;
 - confirmação;
 - custo;
 - receiving.
-

FASE 8 — Mobile

Criar:

- login;
 - scanner;
 - consulta;
 - recebimento;
 - movimentação;
 - inventário.
-

FASE 9 — Orders

Criar:

- pedidos;
- itens;

- reservas;
 - status.
-

FASE 10 — Picking

Criar:

- picking;
 - FEFO;
 - scanner;
 - conferência;
 - expedição.
-

FASE 11 — Realtime

Adicionar SignalR.

FASE 12 — Analytics

Adicionar:

- dashboard;
 - curva ABC;
 - giro;
 - aging;
 - validade;
 - ocupação.
-

FASE 13 — Recursos avançados

- mapa 2D;
 - mapa 3D;
 - rotas;
 - heatmap;
 - otimizações.
-

136. Divisão sugerida do grupo

Caso sejam quatro integrantes:

Backend / Domain

Responsável principalmente por:

- Domain;
- regras;
- inventory;
- API.

Frontend

Responsável por:

- React;
- UX;
- dashboard;
- administração.

Mobile

Responsável por:

- React Native;
- scanner;
- fluxos operacionais.

Data / Integration / DevOps

Responsável principalmente por:

- PostgreSQL;
- NF-e;
- Docker;
- CI/CD;
- analytics.

Isso não significa isolamento.

Todos devem entender o sistema.

137. Regra de integração

Nenhum integrante poderá construir durante meses algo isolado para só integrar no final.

O fluxo será incremental:


```
Backend
↓
Teste
↓
Frontend
↓
Teste integrado
↓
Commit
```

138. Definition of Done de uma funcionalidade

Uma tarefa somente estará concluída quando:

- código implementado;
- validações implementadas;
- tratamento de erros;
- autorização;
- testes;
- integração com banco;
- UI quando necessária;
- documentação;
- revisão;
- CI passando.

139. Política de desenvolvimento

Prioridade:

```
CORRETO
↓
TESTADO
↓
SEGURO
↓
LEGÍVEL
↓
PERFORMÁTICO
↓
BONITO
```

Não inverter essa ordem.

140. O que não faremos

Evitar:

100 funcionalidades
20% funcionando

Preferir:

20 funcionalidades
100% funcionando

141. Riscos técnicos

Escopo excessivo

Mitigação:

fases independentes.

NF-e

Mitigação:

módulo isolado.

Concorrência

Mitigação:

transações e testes.

Mobile

Mitigação:

online-first.

3D

Mitigação:

fase final.

Integração do grupo

Mitigação:

Pull Requests pequenos.

142. Critérios de sucesso

Ao final, uma demonstração ideal deverá permitir:

1. Gestor entra na Web.
2. Importa uma NF-e.
3. Confere produtos.
4. Confirma recebimento.
5. Estoquista recebe tarefa.
6. Escaneia item.
7. Escaneia localização.
8. Material entra.
9. Kardex é registrado.
10. Dashboard atualiza.
11. Pedido é criado.
12. Estoque é reservado.
13. Picking é gerado com FEFO.
14. Operador escaneia produto.
15. Sistema valida.
16. Pedido é expedido.
17. Saldo é atualizado.
18. Histórico completo permanece disponível.

Esse será o principal fluxo de demonstração.

143. Diferenciais de portfólio

O projeto demonstrará conhecimento em problemas muito além de CRUD:

- concorrência;
- idempotência;
- transações;
- auditoria;
- ledger;
- custo;
- integração fiscal;
- scanner;
- WebSockets;
- algoritmos;
- CI/CD;
- Docker;

- segurança;
 - testes automatizados;
 - arquitetura.
-

144. Documentos que deverão existir

Além do README:

```
README.md
CONTRIBUTING.md
SECURITY.md
CHANGELOG.md
LICENSE

docs/
├─ requirements.md
├─ business-rules.md
├─ architecture.md
├─ database.md
├─ nfe.md
├─ deployment.md
├─ testing.md
└─ adr/
```

145. Primeiro passo real

Não começaremos criando Controllers ou telas.

A sequência inicial deverá ser:

1. Definir nome
 2. Criar requisitos
 3. Modelar domínio
 4. Criar ERD
 5. Criar arquitetura
 6. Criar GitHub
 7. Configurar ambiente
 8. Criar backend
 9. Criar banco
 10. Criar primeiro módulo
-

146. Regra principal do projeto

O projeto deverá ser desenvolvido como se pudesse futuramente ser utilizado por uma empresa real.

Porém:

Não construiremos complexidade que ainda não possuímos necessidade real de utilizar.

Isso será o princípio central da arquitetura.

README OFICIAL DO REPOSITÓRIO

O conteúdo abaixo será utilizado como base para o `README.md`.



WMS Inteligente

Sistema corporativo de gestão de estoque, almoxarifado e operações logísticas desenvolvido com .NET, React, React Native e PostgreSQL.

O **WMS Inteligente** é uma plataforma completa para gerenciamento de materiais e operações dentro de almoxarifados e centros de distribuição.

O projeto integra uma aplicação Web destinada à administração e gestão com um aplicativo Mobile voltado aos operadores do estoque, permitindo controlar todo o fluxo de materiais desde o recebimento até a expedição.

O objetivo é desenvolver uma solução com arquitetura e regras de negócio próximas às encontradas em aplicações corporativas reais, abordando problemas como concorrência, rastreabilidade, inventário, lotes, validade, reservas, custos, NF-e e operações através de códigos de barras.



Objetivo

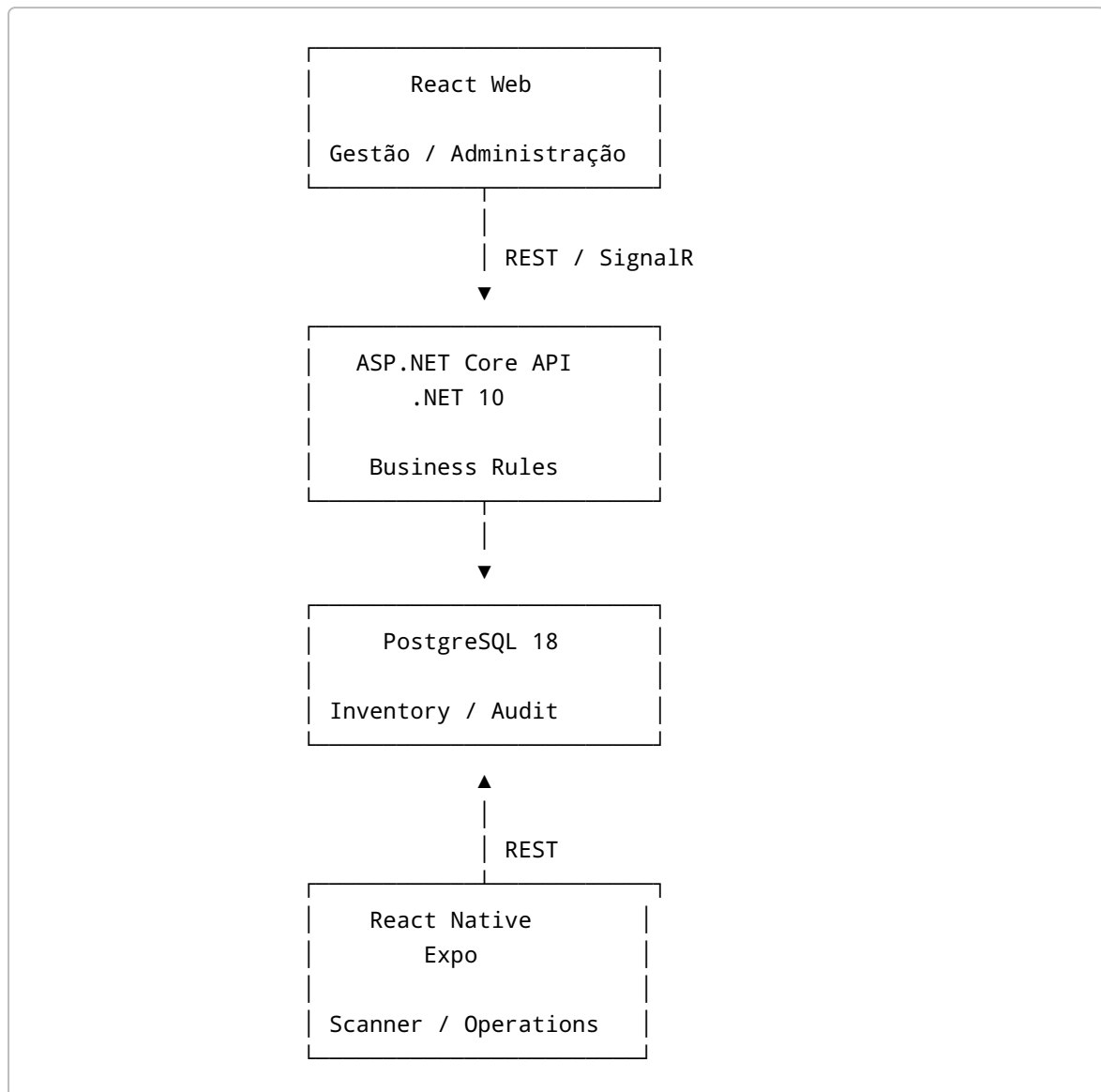
Centralizar e automatizar processos de estoque que normalmente dependem de planilhas, conferências manuais e sistemas pouco integrados.

A plataforma busca proporcionar:

- rastreabilidade completa;
- redução de divergências;
- controle de lotes e validades;
- localização física de produtos;

- inventários mais eficientes;
- prevenção de estoque negativo;
- controle de reservas;
- gestão de recebimentos;
- picking orientado;
- atualização em tempo real;
- informações gerenciais.

Arquitetura



O backend utiliza uma abordagem de **Modular Monolith + Clean Architecture**, mantendo os diferentes domínios do sistema desacoplados sem introduzir prematuramente a complexidade operacional de microserviços.

Tecnologias

Backend

C#

.NET

PostgreSQL

- C#
- .NET 10 LTS
- ASP.NET Core
- Entity Framework Core
- PostgreSQL
- Npgsql
- FluentValidation
- ASP.NET Core Identity
- JWT
- SignalR
- OpenAPI
- OpenTelemetry

Frontend

React

TypeScript

Tailwind

- React
- TypeScript
- Vite
- Tailwind CSS
- shadcn/ui
- TanStack Query
- React Hook Form
- Zod
- Recharts

Mobile

React Native

Expo

- React Native
- Expo

- TypeScript
- Expo Router
- expo-camera
- expo-haptics
- expo-secure-store
- TanStack Query

Infraestrutura

Docker

GitHub Actions

- Docker
- Docker Compose
- GitHub Actions
- PostgreSQL
- Structured Logging
- Health Checks

Principais módulos

Identity
Catalog
Suppliers
Warehouse
Inventory
Receiving
NFe
Orders
Picking
InventoryCount
Costing
Audit
Notifications
Analytics

Segurança

O sistema será desenvolvido considerando:

- autenticação JWT;
- refresh tokens;
- RBAC;
- permissões granulares;
- rate limiting;

- validação de entrada;
- armazenamento seguro de credenciais;
- auditoria de operações;
- proteção contra estoque negativo;
- idempotência;
- controle de concorrência.

Controle de Estoque

O estoque utiliza duas estruturas complementares.

Inventory Ledger

Mantém o histórico imutável de todas as movimentações.

```
+100 RECEBIMENTO
-20  PICKING
-2   AJUSTE
-----
78   SALDO
```

Stock Balance

Mantém o saldo atual otimizado.

```
On Hand:    100
Reserved:    25
Available:   75
```

Essa combinação oferece desempenho sem perder rastreabilidade.

Endereçamento

O sistema modela fisicamente o armazém:

```
Warehouse
  ↓
Zone
  ↓
Aisle
  ↓
Rack
  ↓
Level
```

↓
Bin

Exemplo:

A-03-07-02-04

Aplicativo Mobile

O aplicativo é destinado aos operadores do estoque.

Principais operações:

- leitura de código de barras;
- consulta de produtos;
- recebimento;
- armazenagem;
- transferências;
- picking;
- inventários.

O scanner utiliza feedback visual, sonoro e tátil para reduzir erros operacionais.

NF-e

O sistema possuirá um módulo para importação de XML de NF-e.

Fluxo:

```
XML
↓
Validação
↓
Parser
↓
Fornecedor
↓
Itens
↓
Mapeamento de Produtos
↓
Conferência
↓
Recebimento
```

A importação não movimenta estoque automaticamente antes da conferência do usuário.

Lotes e validade

Produtos poderão possuir controle de:

- lote;
- fabricação;
- validade.

Para produtos perecíveis, o sistema poderá aplicar:

FEFO

First Expired, First Out

Priorizando durante a separação os lotes com vencimento mais próximo.

Inventário

Suporte planejado para:

- inventário geral;
 - inventário rotativo;
 - contagem por endereço;
 - contagem por produto;
 - contagem cega;
 - recontagem;
 - análise de divergências.
-

Atualização em tempo real

Através do **SignalR**, movimentações realizadas pelo aplicativo poderão atualizar imediatamente a interface administrativa.

```
Mobile
  ↓
API
  ↓
Movimentação
  ↓
PostgreSQL
  ↓
SignalR
```

↓
Dashboard Web

Sem necessidade de atualizar manualmente a página.

Analytics

Indicadores planejados:

- valor do estoque;
- Curva ABC;
- giro;
- aging;
- produtos abaixo do mínimo;
- ruptura;
- produtos próximos ao vencimento;
- ocupação;
- movimentações;
- produtividade operacional.

Testes

O projeto terá:

- Unit Tests;
- Integration Tests;
- Architecture Tests;
- testes de componentes;
- testes E2E;
- testes de concorrência.

Cenários críticos incluem:

- ✓ impedir estoque negativo
- ✓ impedir NF-e duplicada
- ✓ impedir reserva superior ao saldo
- ✓ detectar item incorreto no picking
- ✓ impedir operações sem autorização
- ✓ impedir duplicação causada por retry
- ✓ garantir consistência em operações concorrentes

Docker

A infraestrutura poderá ser iniciada através de Docker Compose.

```
Docker Compose
|
├── PostgreSQL
├── ASP.NET Core API
└── React Web
```

Estrutura

```
/
├── backend/
│   ├── src/
│   │   ├── Wms.Api/
│   │   ├── Wms.Application/
│   │   ├── Wms.Domain/
│   │   └── Wms.Infrastructure/
│   └── tests/
├── web/
├── mobile/
├── infra/
├── docs/
├── scripts/
├── .github/
├── docker-compose.yml
└── README.md
```

Roadmap

Fase 0 — Engenharia

- ☐ Requisitos
- ☐ Regras de negócio
- ☐ Arquitetura
- ☐ Banco
- ☐ Diagramas
- ☐ ADRs

Fase 1 — Fundação

- ☐ Backend
- ☐ Frontend
- ☐ PostgreSQL
- ☐ Docker
- ☐ CI

Fase 2 — Segurança

- ☐ Usuários
- ☐ Login
- ☐ Roles
- ☐ Permissões
- ☐ JWT
- ☐ Refresh Tokens

Fase 3 — Catálogo

- ☐ Produtos
- ☐ Categorias
- ☐ Unidades
- ☐ Barcodes
- ☐ Fornecedores

Fase 4 — Armazém

- ☐ Warehouses
- ☐ Zones
- ☐ Aisles
- ☐ Racks
- ☐ Levels
- ☐ Bins

Fase 5 — Estoque

- ☐ Lotes
- ☐ Stock Balance
- ☐ Inventory Ledger
- ☐ Entradas
- ☐ Saídas
- ☐ Transferências
- ☐ Ajustes
- ☐ Reservas
- ☐ Concorrência
- ☐ Idempotência

Fase 6 — Inventário

- ☐ Contagem
- ☐ Contagem cega
- ☐ Recontagem
- ☐ Divergências
- ☐ Ajustes

Fase 7 — NF-e

- ☐ Upload XML
- ☐ Parser
- ☐ Product Matching
- ☐ Recebimento

- ☐ Custo Médio

Fase 8 — Mobile

- ☐ Login
- ☐ Scanner
- ☐ Consulta
- ☐ Recebimento
- ☐ Movimentação
- ☐ Inventário

Fase 9 — Pedidos

- ☐ Orders
- ☐ Reservations
- ☐ Status

Fase 10 — Picking

- ☐ Picking
- ☐ FEFO
- ☐ Scanner
- ☐ Conferência
- ☐ Expedição

Fase 11 — Realtime

- ☐ SignalR
- ☐ Alertas
- ☐ Dashboard em tempo real

Fase 12 — Analytics

- ☐ Dashboard
- ☐ Curva ABC
- ☐ Giro
- ☐ Aging
- ☐ Ocupação

Fase 13 — Advanced WMS

- ☐ Mapa 2D
- ☐ Mapa 3D
- ☐ Heatmap
- ☐ Roteirização de Picking



Evoluções futuras

Possíveis evoluções:

- otimização automática de slotting;

- algoritmos de roteirização;
 - previsão de demanda;
 - previsão de ruptura;
 - sugestão automática de compras;
 - detecção de anomalias;
 - operação offline;
 - integrações com ERP;
 - leitores industriais;
 - etiquetas;
 - impressão térmica;
 - múltiplos centros de distribuição.
-

Objetivo acadêmico e profissional

Além de resolver um problema empresarial real, o projeto foi concebido para aplicar conhecimentos modernos de Engenharia de Software, incluindo:

- arquitetura;
 - APIs;
 - segurança;
 - concorrência;
 - transações;
 - bancos relacionais;
 - desenvolvimento Web;
 - desenvolvimento Mobile;
 - testes;
 - DevOps;
 - integração contínua;
 - observabilidade.
-

Equipe

Projeto desenvolvido em grupo por estudantes de **Análise e Desenvolvimento de Sistemas**.

Status

 **Em planejamento e desenvolvimento.**

O projeto será desenvolvido incrementalmente, priorizando estabilidade, testes e integração antes da implementação de funcionalidades avançadas.

Licença

A licença do projeto será definida pela equipe antes da primeira versão pública.

WMS Inteligente
Sistema de Gestão de Estoque e Operações Logísticas